

Assignment 1B — Domain LLM Adaptation & Production Optimization

End-to-End Pipeline: Domain Data → QLoRA Fine-Tuning → Quantization & Speculative Decoding → Production Economics

Component	Part	Marks
Domain Data Collection & Instruction Dataset	Part A	2
QLoRA Instruction Fine-Tuning	Part B	5
Inference Optimization & Production Metrics	Part C	8
TOTAL		15

Completing this pipeline — from domain PDFs to a fine-tuned, quantized, production-benchmarked domain LLM — gives students direct, hands-on experience with the adaptation and serving workflow used by AI engineering teams building real-world LLM products.

Domain & Model Selection

Select one domain from the table below. The same domain corpus is used across all three parts. Choose the model based on the GPU available to you. Use the tokenizer of your chosen model throughout — do not mix tokenizers across parts.

T4 GPU (16 GB VRAM) — available on free Colab. A100 GPU / L40S — BITS remote lab.

Both model choices use the same assignment pipeline; only scale differs.

The following table is for your reference. You are free to choose any domain and any model.

Use Case / Domain	T4 GPU — Choice 1	T4 GPU — Choice 2	A100 GPU — Choice 1	A100 GPU — Choice 2
Medical & Clinical Literature	BioGPT-Large (347M) microsoft/biogpt-large	GPT-2-Medium (345M) openai-community/gpt2-medium	BioMedLM (2.7B) stanford-crfm/BioMedLM	Mistral-7B-v0.1 (7B) mistralai/Mistral-7B-v0.1
Legal Documents & Contracts	SmolLM2-360M (360M) HuggingFaceTB/SmolLM2-360M	GPT-2-Medium (345M) openai-community/gpt2-medium	Mistral-7B-v0.1 (7B) mistralai/Mistral-7B-v0.1	Qwen2.5-7B (7B) Qwen/Qwen2.5-7B
Financial Reports & Filings	SmolLM2-360M (360M) HuggingFaceTB/SmolLM2-360M	TinyLlama-1.1B (1.1B) TinyLlama/TinyLlama-1.1B-...	Qwen2.5-7B (7B) Qwen/Qwen2.5-7B	Mistral-7B-v0.1 (7B) mistralai/Mistral-7B-v0.1
Manufacturing & Technical Standards	TinyLlama-1.1B (1.1B) TinyLlama/TinyLlama-1.1B-...	SmolLM2-360M (360M) HuggingFaceTB/SmolLM2-360M	Mistral-7B-v0.1 (7B) mistralai/Mistral-7B-v0.1	LLaMA-3-8B (8B) meta-llama/Meta-Llama-3-8B
HR Policy & Corporate Documents	SmolLM2-1.7B (1.7B) HuggingFaceTB/SmolLM2-1.7B	TinyLlama-1.1B (1.1B) TinyLlama/TinyLlama-1.1B-...	Gemma-7B (7B) google/gemma-7b	LLaMA-3-8B (8B) meta-llama/Meta-Llama-3-8B
Scientific Research Papers	SmolLM2-360M (360M) HuggingFaceTB/SmolLM2-360M	GPT-2-Large (774M) openai-community/gpt2-large	LLaMA-3-8B (8B) meta-llama/Meta-Llama-3-8B	Qwen2.5-7B (7B) Qwen/Qwen2.5-7B

Always load the tokenizer paired with your chosen model using `AutoTokenizer.from_pretrained('<model-id>')`. Do not swap tokenizers between models — vocabulary alignment breaks fine-tuning.

Detailed inferences and justification for all the tasks are mandatory.

PART A — Domain Data Collection & Instruction Dataset [2 Marks]

Build a small domain corpus from PDFs and convert it into clean **.txt** files. You will reuse this same cleaned corpus in **Part B (B1)** to create your instruction dataset and in **Part C** for inference benchmarking.

Step 1 — Domain Data Collection & Cleaning [1 Mark]

Collect domain-specific PDF documents (minimum 5). Extract text **page-by-page** and save each document as a separate **.txt** file inside a folder named **domain_corpus/**.

- **Length filter** — remove documents (or extracted files) that are too short to be useful (choose and justify a minimum length threshold).
- **Deduplication** — remove near-duplicate files or repeated pages/sections (state the method used, e.g., hashing or similarity threshold).
- **Language filter** — retain only English-language documents (mention the library/tool used for language detection).
- Report corpus statistics **before and after** each cleaning step (e.g., #PDFs, #.txt files, total pages extracted). Identify which step reduced the corpus the most and briefly explain why.

Step 2 — Baseline Model Output [1 Mark]

Load your chosen base model in **bfloat16**. Run it on **3 domain-specific prompts** (no system prompt) and save the generated outputs. This is your **pre-adaptation baseline**; you will compare it against your fine-tuned adapter outputs in **Part B3**.

- **Architecture report** — print total parameter count, number of decoder layers, hidden size, and vocabulary size
- **Save baseline** — save the 3 baseline outputs as a markdown cell or CSV — they will be reused in Part B Step 3

Tip: You do not need tokenization, sequence packing, or CPT training in this assignment. The cleaned .txt files feed directly into Part B's instruction dataset creation step.

PART B — Instruction Fine-Tuning with QLoRA [5 Marks]

Fine-tune your base model using **QLoRA** on an instruction dataset derived from your **cleaned domain corpus** (Part A). Use the same tokenizer and base model family throughout. If you have access to an A100/L40S, you may scale up to a larger model, but keep the domain and evaluation prompts consistent.

Part B1 — Instruction Dataset Creation [2 Marks]

Transform your cleaned domain corpus into a structured instruction dataset suitable for supervised fine-tuning.

- **Source** — use only the cleaned **.txt** files from Part A (Step 1). Choose a dataset size appropriate to your model/GPU and justify your choice (e.g., number of examples and average tokens per example).
- **Format** — JSONL; each row must include **instruction** (user question) and **response** (answer grounded in your domain text). Do not add external facts beyond what is in the corpus.
- **Method** — you may use heuristic/manual generation or synthetic generation via an external LLM. If you use an LLM, include the **exact prompt template** and show **5 sample** instruction-response pairs.
- **Split** — split into **80% train** and **20% evaluation** using a fixed random seed; report the example counts for both.

Synthetic generation tip: 'Read the text below and generate 10 instruction-response pairs in JSON format based ONLY on this text. Each entry must have instruction and response keys.'

Part B2 — QLoRA Fine-Tuning with One Adapter Configurations [2 Marks]

Apply QLoRA using **transformers**, **peft**, and **bitsandbytes** (4-bit base model). Train **any one** of the adapter configurations below on your instruction training set using an SFT setup that matches your model's chat/instruction format (use the model's chat template where applicable). Report your key training hyperparameters (batch size, learning rate, epochs/steps, max sequence length).

Adapter	LoRA Rank (r)	LoRA Alpha	Target Modules	Expected Effect
Adapter A (Low Capacity)	8	16	q_proj, v_proj	Faster; may underfit complex domain queries
Adapter B (Balanced)	16	32	q_proj, v_proj	Good quality / cost trade-off
Adapter C (High Capacity)	32	32	q_proj, v_proj, o_proj	Best quality; slower; higher VRAM usage

Part B3 — Evaluation & Comparative Analysis [1 Mark]

Run your trained adapter on the **same 3 domain prompts** used in Part A (Step 2). Present a **side-by-side** comparison (baseline vs adapter) and describe what improved or degraded (e.g., correctness, domain terminology usage, completeness, hallucinations).

PART C — Inference Optimization & Production Metrics [8 Marks]

Apply three inference techniques to your domain model and analyse their effect on **output quality/diversity**, **latency/throughput**, and **cost**. Keep the prompts fixed across runs so differences are attributable to inference settings. Use the same base model loaded in Part A throughout this part.

Step 1 — Decoding Strategies & Sampling [3 Marks]

Decoding strategy controls how the model selects the next token at each generation step. Different strategies produce radically different outputs from the exact same model and prompt — understanding this trade-off is essential for production domain assistants.

Implement all five configurations below and run each on the same **3 domain prompts** (use the prompts from Part B3). Use **max_new_tokens=150** for all runs. For each run, record (1) the generated text, and (2) a simple speed metric (e.g., tokens/sec or total generation time).

Strategy	When to Use
Greedy Decoding	Fastest; deterministic; best for factual domain lookups where one correct answer exists
Beam Search	More coherent than greedy; higher quality for formal outputs (reports, summaries)
Top-K Sampling	Introduces diversity; good for creative or explanatory domain content
Top-P (Nucleus) Sampling	Adapts vocabulary size per step; recommended for open-ended domain Q&A
Temperature Scaling	Low temp → conservative/safe; high temp → diverse/creative; compare both

Tip: What to vary: for Temperature Scaling, run the same prompt at different temperature. For all other strategies, one run per prompt is sufficient.

Fill the comparison table for each strategy on your 3 domain prompts:

Prompt	Greedy	Beam Search	Top-K	Top-P	Temp 1	Temp 2	Temp 3
Domain Prompt 1	[fill]	[fill]	[fill]	[fill]	[fill]		[fill]

Domain Prompt 2	[fill]	[fill]	[fill]	[fill]	[fill]		[fill]
Domain Prompt 3	[fill]	[fill]	[fill]	[fill]	[fill]		[fill]

Write a 100-word analysis: which decoding strategy would you deploy for your domain use case, and why? Cite specific examples from your output table to support your answer.

Domain tip: factual domains (legal, medical, compliance) typically benefit from Greedy or low-temperature sampling because consistency and accuracy matter more than diversity. Open-ended domains (research, HR, creative writing) benefit from Top-P or moderate temperature because varied phrasing improves usefulness.

Step 2 — Speculative Decoding [2 Marks]

Select a **draft model** smaller than your target model and run speculative decoding. Both models must fit in VRAM simultaneously. Benchmark speculative decoding on the same prompts used in Step 1 and report throughput/latency change plus any noticeable quality differences.

GPU	Target Model	Recommended Draft Model	HuggingFace Draft ID
T4 (16 GB)	SmolLM2-360M or TinyLlama-1.1B	SmolLM2-135M	HuggingFaceTB/SmolLM2-135M-Instruct
T4 (16 GB)	GPT-2-Medium (345M)	GPT-2 (124M)	openai-community/gpt2
A100 (40 GB)	Mistral-7B / LLaMA-3-8B	SmolLM2-1.7B	HuggingFaceTB/SmolLM2-1.7B-Instruct

Present your observations.

Step 3 — 4-bit Quantization & Production Cost [3 Marks]

Apply **4-bit quantization** to your base model and measure the change in **peak VRAM** and **throughput**. Then compute and summarise the **cost per 1M tokens** for each configuration measured in Part C (baseline bfloat16, 4-bit, and 4-bit + speculative decoding).

- **Load** — use `BitsAndBytesConfig(load_in_4bit=True, bnb_4bit_quant_type='nf4', bnb_4bit_compute_dtype=torch.bfloat16)`
- **Benchmark** — run the same 10 domain prompts with your best decoding strategy from Step 1; record VRAM and throughput
- **Cost formula** — $\text{Cost} / 1\text{M tokens} = (1,000,000 \div \text{throughput_tok_per_sec}) \div 3600 \times \text{hourly_rate}$
[₹3.50/hr for T4 | ₹12.00/hr for A100]

Configuration	Peak VRAM (GB)	Throughput (tok/s)	Cost / 1M tokens (₹)
bfloat16 (Part A baseline)	[fill]	[fill]	[fill]
4-bit NF4 quantized	[fill]	[fill]	[fill]

4-bit + Speculative Decoding (Step 2)	<i>[fill]</i>	<i>[fill]</i>	<i>[fill]</i>
---------------------------------------	---------------	---------------	---------------

Write 2–3 sentences recommending which configuration you would deploy for your domain, citing the measured cost and any quality trade-off observed.

Submission Deliverables

File	Contents
Python Notebook with Output	Includes Part A–C work: data collection/cleaning results, baseline outputs, instruction dataset creation, QLoRA training, decoding comparisons, speculative decoding benchmark, 4-bit quantization benchmark, and cost calculations. Detailed inferences and observations are mandatory. Include your final prompts used for baseline and evaluation, plus key hyperparameters and benchmarking settings.
HTML file of .ipynb with Output	Exported HTML version of the notebook with outputs visible.
instruction_dataset.jsonl	Final JSONL used for fine-tuning, with instruction and response fields.
domain_corpus/*.txt	Cleaned domain text files produced in Step 1

Optional Extension (No Marks)

Load your best fine-tuned adapter (from Part B3) on top of the 4-bit quantized model and re-run the speculative decoding benchmark. This is the realistic production configuration: fine-tuned adapter + 4-bit quantization + speculative decoding combined. Measure whether quality, throughput, and cost change compared to the base model versions from Part C.